Analiza Algoritmilor - final recap.

I Mașini Turing

Niște mașini turing relevante de prim curs sunt:

- mașina **isEven** → verifică dacă imputul este par.

	0	1	□
2_1	$2_1, 0, \rightarrow$	$2_1, 1, \rightarrow$	$2_2, \square, \leftarrow$
2_2	$Y, 0, -$	$N, 1, -$	$N, \square, -$

- mașina **sameFirstLast** → verifică dacă imputul începe și se termină cu aceeași cifră.

	0	1	□
start	first_0, 0, →	first_1, 1, →	Y, □, -
first_0	first_0, 0, →	first_0, 1, →	expect_0, □, ←
first_1	first_1, 0, →	first_1, 1, →	expect_1, □, ←
expect_0	Y, 0, -	N, 1, -	N, □, -
expect_1	N, 0, -	Y, 1, -	N, □, -

- mașina **isPalindrome** → verifică dacă imputul este palindrom.

	0	1	□
start	mem0, □, →	mem1, □, →	Y, □, -
mem0	mem0, 0, →	mem1, 1, →	expect0, □, ←
mem1	mem1, 0, →	mem1, 1, →	expect1, □, ←
expect0	reset, □, ←	N, 1, -	Y, □, -
expect1	N, 0, -	reset, □, ←	Y, □, -
reset	reset, 0, ←	reset, 1, ←	start, □, →

Scanned with CamScanner

- mașina **increment**, care incrementează inputul

	0	1	□
q_1	$q_1, 0, \rightarrow$	$q_1, 1, \rightarrow$	$q_2, \square, \leftarrow$
q_2	$H, 1, -$	$q_2, 0, \leftarrow$	$H, 1, -$

- mașina **sameNumber1And0** → verifică dacă inputul are același număr de 0 și 1.

	0	1	□	X
start	find1, X, →	find0, X, →	Y, □, -	start, X, →
find0	reset, X, ←	find0, 1, →	N, □, -	find0, X, →
find1	find1, 0, →	reset, X, ←	N, □, -	find1, X, →
reset	reset, 0, ←	reset, 1, ←	start, □, →	reset, X, ←

Laborator

(1.) Scrieți o mașină Turing care primește un șir liniar și verifică dacă începe și se termină cu simboluri distincte.

	0	1	□
start	first0, 0, →	first1, 1, →	N, □, -
first0	first0, 0, →	first0, 1, →	expect1, □, ←
first1	first1, 0, →	first1, 1, →	expect0, □, ←
expect0	Y, 0, -	N, 1, -	N, □, -
expect1	N, 0, -	Y, 1, -	N, □, -

Scanned with CamScanner

(2.) Scrieți o mașină Turing care primește un șir liniar și verifică dacă are lungime impară și simbolul din mijloc este 0.

	0	1	□
start	last0, □, →	last1, □, →	N, □, -
last0	last0, 0, →	last0, 1, →	erase0, □, ←
last1	last1, 0, →	last1, 1, →	erase1, □, ←
erase0	reset, □, ←	reset, □, ←	Y, □, -
erase1	reset, □, ←	reset, □, ←	N, □, -
reset	reset, 0, ←	reset, 1, ←	start, □, →

(3.) Scrieți o mașină Turing care primește un șir liniar și lasă pe bandă complementul lui.

	0	1	□
2	2, 1, →	2, 0, →	H, □, -

(4.) Scrieți o mașină Turing care curăță toată banda (atât la stânga cât și la dreapta) și apoi lasă scris doar "1".

	0	1	□
left	left, 0, ←	left, 1, ←	right, □, →
right	right, □, →	right, □, →	H, 1, -

Scanned with CamScanner

(5.) Scrieți o mașină Turing care inversează cuvântul primit pe bandă.

	0	1	□	X
addX	addX, 0, →	addX, 1, →	firstX, X, ←	N, X, -
firstX	lastX, 0, ←	lastX, 1, ←	eraseX, □, →	firstX, X ←
lastX	Keep0, 0, →	Keep1, 1, →	eraseX, □, →	firstX, X, ←
eraseX	eraseX, 0 ←	eraseX, 1, ←	Y, □, -	eraseX, -, →
keep0	keep0, 0, →	keep0, 1, →	lastX, 0, ←	keep0, X, →
keep1	keep1, 0, →	keep1, 1, →	lastX, 1, ←	keep1, X, →

Computing – Lab

(1.) Scrieți configurațiile mașinii isEven pentru imputul:

a) 100 ; b) 1011

	0	1	□
q_1	$q_1, 0, \rightarrow$	$q_1, 1, \rightarrow$	$q_2, □, \leftarrow$
q_2	Y, 0, -	N, 1, -	N, □, -

a) $(\varepsilon, 100, q_1) \vdash_M (1, 00, q_1) \vdash_M (10, 0, q_1) \vdash_M (100, □, q_1) \vdash_M$
$\vdash_M (10, 0, q_2) \vdash_M (10, 0, Y)$

b) $(\varepsilon, 1011, q_1) \vdash_M (1, 011, q_1) \vdash_M (10, 11, q_1) \vdash_M (101, 1, q_1) \vdash_M$
$\vdash_M (1011, □, q_1) \vdash_M (101, 1, q_2) \vdash_M (101, 1, N)$

(2) Scrieți primele 15 configurații prin care trece mașina de mai jos pentru imputul 1011. Ce observați?

	0	1	□
q_1	$q_1, 1, \rightarrow$	$q_1, 0, \rightarrow$	$q_2, □, \leftarrow$
q_2	$q_2, 1, \rightarrow$	$q_2, 0, \rightarrow$	$q_1, □, \rightarrow$

Scanned with CamScanner

$(\varepsilon, 1011, q_1) \vdash_M (0, 011, q_1) \vdash_M (01, 11, q_1) \vdash_M (010, 1, q_1) \vdash_M$
$\vdash_M (0100, \square, q_1) \vdash_M (010, 0, q_2) \vdash_M (0101, \square, q_2) \vdash_M (0101\square, \square, q_1) \vdash_M$
$\vdash_M (0101, \square, q_2) \vdash_M (0101\square, \square, q_1) \vdash_M (0101, \square, q_2) \vdash_M (0101\square, \square, q_1)$
$\vdash_M (0101, \square, q_2) \vdash_M (0101\square, \square, q_1) \vdash_M (0101, \square, q_2) \vdash_M (0101\square, \square, q_1)$

Se observă că mașina ciclează pentru acest input.

(3) Scrieți primele 15 configurații prin care trece mașina de mai jos pentru inputul 01.

	0	1	$\square$
q_1	$q_1, 0, \rightarrow$	$q_1, 1, \rightarrow$	$q_2, 1, \leftarrow$
q_2	$q_2, 0, \leftarrow$	$q_2, 1, \leftarrow$	$q_1, 1, \rightarrow$

$(\varepsilon, 01, q_1) \vdash_M (0, 1, q_1) \vdash_M (01, \square, q_1) \vdash_M (0, 11, q_2) \vdash_M$
$\vdash_M (\square, 011, q_2) \vdash_M (\square, \square 011, q_2) \vdash_M (1, 011, q_1) \vdash_M (10, 11, q_1) \vdash_M$
$\vdash_M (101, 1, q_1) \vdash_M (1011, \square, q_1) \vdash_M (101, 11, q_2) \vdash_M (10, 111, q_2)$
$\vdash_M (1, 0111, q_2) \vdash_M (\square, 10111, q_2) \vdash_M (\square, \square 10111, q_2)$

Reduceri Turing

Fie $f: \Sigma^* \rightarrow \{\text{FALSE}, \text{TRUE}\}$

O mașină Turing M **decide** o problemă f dacă:
$\forall w \in \Sigma^*, M[w] \rightarrow \text{TRUE} \Rightarrow f(w) = \text{TRUE}$
$\forall w \in \Sigma^*, M[w] \rightarrow$ ~~TRUE~~ FALSE $\Rightarrow f(w) =$ FALSE

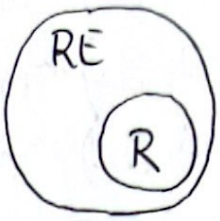
O funcție f pentru care există o mașină Turing M care o decide se numește decidabilă.

O mașină M **acceptă** o problemă f $\overset{\text{def}}{\Leftrightarrow} \forall w \in \Sigma^*$, $f(w) = \text{TRUE} \Leftrightarrow M[w] \rightarrow \text{TRUE}$

Scanned with CamScanner

O funcție f pentru care există o mașină Turing M care o acceptă se numește acceptabilă.

$R \overset{def}{=} \{f \mid f \text{ este decidabilă}\}$
$RE \overset{def}{=} \{f \mid f \text{ este acceptabilă}\}$



$t : \Sigma_1^* \to \Sigma_2^*$

O mașină M <u>computează</u> o funcție t $\overset{def}{=}$ $\forall w \in \Sigma_1^*$, $M[w] \to t(w)$.

O funcție t pentru care există o mașină Turing M care o computează se numește computabilă.

Probleme nedecidabile clasice

1. **Halting problem.**

$$HALT(enc(M,w)) = \begin{cases} TRUE, & M[w] \text{ se termină} \\ FALSE, & \text{în caz contrar.} \end{cases}$$

2. **Co-Halting problem.**

$$\overline{HALT}(enc(M,w)) = \begin{cases} TRUE, & M[w] \text{ nu se termină} \\ FALSE, & \text{în caz contrar.} \end{cases}$$

3. **ALL problem.**

$$ALL(enc(M)) = \begin{cases} TRUE; & \forall w \in \Sigma^*, M[w] \text{ se termină} \\ FALSE; & \text{în caz contrar.} \end{cases}$$

4. **ANY problem**

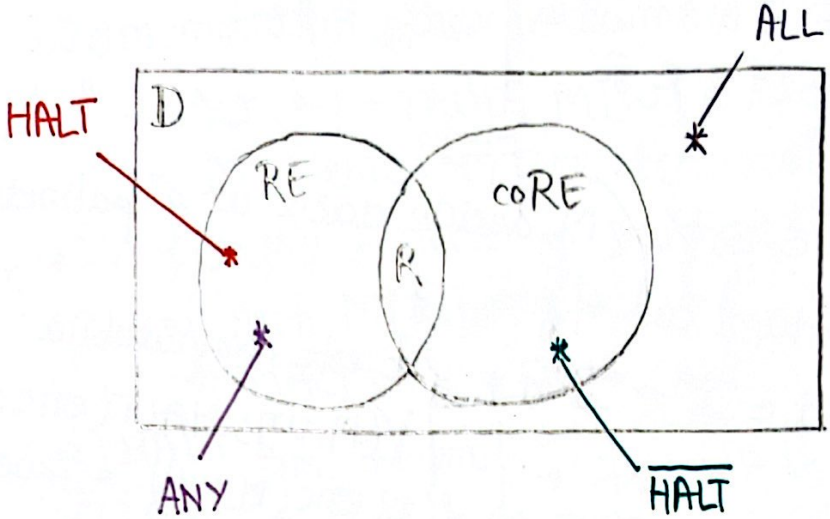
$$ANY(enc(M)) = \begin{cases} TRUE; & \exists w \in \Sigma^*, M[w] \text{ se termină} \\ FALSE; & \text{în caz contrar.} \end{cases}$$

Scanned with CamScanner

$f \leq_m g \overset{def}{\Leftrightarrow} \exists t : \Sigma^* \to \Sigma^*$ a.î. $\begin{cases} t \text{ este computabilă} \\ \forall w \in \Sigma^*, f(w) = g(t(w)) \end{cases}$

Proprietăți

- Dacă $f \leq_m g$
  - $f \notin \overline{R} \Rightarrow g \notin R$
  - $f \notin \overline{RE} \Rightarrow g \notin RE$
  - $g \in \overline{R} \Rightarrow f \in R$
  - $g \in RE \Rightarrow f \in RE$
- $f \leq_m f$
- $f \leq_m g$ și $g \leq_m h \Rightarrow f \leq_m h$



$\mathbb{D} = \{\mathcal{T} \mid \mathcal{T}$ nu este acceptabilă și decidabilă$\}$

Laborator

① Demonstrați, folosind reduceri, că următoarele probleme sunt nedecidabile

- $f_{111}(M) = \text{TRUE} \Leftrightarrow M[111] \to \text{TRUE}$.

Încercăm să arătăm că $f_h \leq_m f_{111}$.

Trebuie să găsim o transformare t care primește inputul lui HALTING PROBLEM și întoarce un input pentru $f_{111} : M, w \to t \to M^*$.

După ce am găsit asta mai rămâne de demonstrat doar $f_h(M, w) = \text{TRUE} \Leftrightarrow f_{111}(M^*) = \text{TRUE}$.

Scanned with CamScanner

$M^*(x)$:
run $M[w]$
return $x == 111$

Demonstrații:

① $f_h(M, w) = TRUE \Rightarrow M[w]$ se termină $\Rightarrow$ vom returna $x == 111 \Rightarrow$ când x este 111, M^* va tranziționa în $Y \Rightarrow$
$\Rightarrow M^*[111] \to TRUE \Rightarrow f_{111}(M^*) = TRUE$. (*)

② $f_{111}(M^*) = TRUE \Rightarrow M^*[111] \to TRUE \Rightarrow$ am ajuns la return $\Rightarrow$ run $M[w]$ nu a ciclat $\Rightarrow M[w]$ s-a oprit $\Rightarrow$
$\Rightarrow f_h(M, w) = TRUE$ (**)

Din (*) și (**) $\Rightarrow \begin{matrix} f_h \leq_m f_{111} \\ f_h \notin \mathcal{R} \end{matrix} \Rightarrow f_{111} \notin \mathcal{R}$.

- $f_p(M) = TRUE \Leftrightarrow \forall w$, M decide dacă w e palindrom

Încercăm să arătăm că $f_h \leq_m f_p$

$HALT \leq_m f \overset{def}{\Leftrightarrow} \exists t \in \Sigma^* \to \Sigma^*$ a.î. $\begin{cases} t \text{ este computabilă} \\ \forall (M, w);\ HALT(enc(M, w)) = \\ = TRUE \Leftrightarrow f(t(enc(M, w))) = TRUE. \end{cases}$

Transformarea t produce mașina:

$M^*[x]$:
run $M[w]$;
check if x is palindrome.

Demonstrăm echivalența cu 2 demonstrații:

"$\Rightarrow$": $HALT(enc(M, w)) = TRUE \Rightarrow M[w]$ se termină $\Rightarrow$
$\Rightarrow$ se verifică dacă x e palindrom; când x este palindrom $\Rightarrow M^*$ va tranziționa în $Y \Rightarrow$
$\Rightarrow f(t(enc(M, w))) = TRUE$.

"$\Leftarrow$": $f(t(enc(M, w))) = TRUE \Rightarrow M^*[x]$ decide dacă x este palindrom pt $\forall x \Rightarrow$ am ajuns la return $\Rightarrow$
$\Rightarrow$ run $M[w]$ nu a ciclat $\Rightarrow M[w]$ s-a oprit $\Rightarrow$
$\Rightarrow HALT(enc(M, w)) = TRUE$

Scanned with CamScanner

- $f_{own}(M) = TRUE \Leftrightarrow M[enc(M)] \to TRUE$

Arătăm că $HALT \leq_m f_{own}$ $\overset{def}{=}$ $\exists t \in \Sigma^* \to \Sigma^*$ a.î.

$\begin{cases} t \text{ computabilă} \\ \forall M,\ HALT(M) = TRUE \Leftrightarrow f(t(M)) = TRUE \end{cases}$

Transformarea t produce mașina

```
M'[x]:
    if x == enc(M')
        run M[w]
    else
        loop.
```

Demonstrăm echivalența în 2 sensuri:

"$\Rightarrow$": $HALT(M) = TRUE$, $M[w]$ se termină $\Rightarrow$
$\Rightarrow M'[enc(M')]$ se termină $\Rightarrow f(t(M)) = TRUE$

"$\Leftarrow$": $f(t(M)) = TRUE \Rightarrow M'[enc(M')]$ se termină $\Rightarrow$
$\Rightarrow$ run $M[w]$ nu a ciclat $\Rightarrow M[w]$ se termină
$\Rightarrow HALT(M) = TRUE$

$\Rightarrow HALT \leq_m f_{own}$, $HALT \notin \mathcal{R} \Rightarrow f_{own} \notin \mathcal{R}$.

- $f_{finite}(M) = TRUE \Leftrightarrow$ M se oprește pentru un număr finit de cuvinte.

Arătăm că $HALT \leq_m f_{finite}$ $\overset{def}{=}$ $\exists t \in \Sigma^* \to \Sigma^*$ a.î.

$\begin{cases} t \text{ computabilă} \\ \forall (M,w),\ HALT(M,w) = TRUE \Leftrightarrow f(t(M,w)) = TRUE \end{cases}$

Transformarea t produce mașina

```
M*(x):
    run M[w] pentru maxim |x| pași
        dacă M[w] s-a oprit
            nu se termină
        else
            se termină
```

Scanned with CamScanner

Demonstrăm echivalența din 2 direcții:

„⇒": HALT(M, w) = TRUE ⇒ M[w] se termină la un moment dat ⇒ M*[x] se oprește ⇒ ⇒ f(t(M, w)) = TRUE.

„⇐": f(t(M, w)) = TRUE ⇒ M*[x] se oprește după un număr finit de cuvinte ⇒ M[w] s-a oprit ⇒ HALT(M, w) = TRUE.

⇒ HALT ≤m f finite, HALT ∉ R ⇒ f finite ∉ R

II Notații asimptotice

- $\Theta(f(n)) = \{g : \mathbb{R}_+ \to \mathbb{R}_+ \mid \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)\}$

(spunem că f și g au aceeași rată de creștere)

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c$; $c \neq 0$; $c \neq \infty$ ⇒ $g \in \Theta(f(n))$

- $O(f(n)) = \{g : \mathbb{R}_+ \to \mathbb{R}_+ \mid \exists c > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, 0 \leq g(n) \leq c \cdot f(n)\}$

(spunem că g crește maxim la fel de repede ca f)

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c$; $c \neq 0$; $c \neq \infty$ ⇒ $g \in O(f(n))$

- $o(f(n)) = \{g : \mathbb{R}_+ \to \mathbb{R}_+ \mid \forall c > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, 0 \leq g(n) \leq c \cdot f(n)\}$

(spunem că g crește strict mai încet decât f)

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$ ⇒ $g \in o(f(n))$

Scanned with CamScanner

- $\Omega(f(n)) = \{g: \mathbb{R}_+ \to \mathbb{R}_+ \mid \exists c>0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0 \quad 0 \leq c \cdot f(n) \leq g(n)\}$

(spunem că g crește mai repede sau la fel de repede ca f)

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = \begin{cases} c \neq 0 \\ \infty \end{cases} \Rightarrow g \in \Omega(f(n))$

- $\omega(f(n)) = \{g: \mathbb{R}_+ \to \mathbb{R}_+ \mid \forall c>0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0 \quad 0 \leq c \cdot f(n) \leq g(n)\}$

(spunem că g crește strict mai repede decât f)

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = \infty \Rightarrow g \in \omega(f(n))$.

ATENȚIE! Relațiile cu limite se aplică <u>doar</u> când f și g sunt funcții monoton crescătoare!

Laborator

① Pentru fiecare din perechile de mai jos, dați un exemplu concret de constante pentru care inegalitățile implicate în definiția clasei de complexitate sunt adevărate.

- $\log(n) \in \Theta(\sqrt{n})$

$f(n) = \sqrt{n}$; $g(n) = \log(n)$

$\Rightarrow \forall c>0, \exists n_0 \in \mathbb{N}$ a.î. $0 \leq \log(n) \leq c\sqrt{n}$, adevărat pentru $c=1$ și $n_0 = 100$

- $100n^2 \in O(2^n)$

$f(n) = 2^n$; $g(n) = 100n^2$

$\Rightarrow \exists c>0, \exists n_0 \in \mathbb{N}$ a.î. $0 \leq 100n^2 \leq c \cdot 2^n$, adevărat pentru $c=100$ și $n_0 = 4$.

- $0{,}001\, n^3 \in \Omega(20\sqrt{n})$

$f(n) = 20\sqrt{n}$; $g(n) = 0{,}001\, n^3$.

Scanned with CamScanner

$\Rightarrow \nexists\, \exists c>0, \exists n_0 \in \mathbb{N}$ a.î. $0 \le c \cdot 20\sqrt{n} \le 0{,}001 \cdot n^2$, adevărat pentru $c = \frac{0{,}001}{20}$ și $n_0 = 1$.

- $10n \in \Theta(n)$

  $f(n) = n$; $g(n) = 10(n)$

  $\Rightarrow \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N}$ a.î. $c_1 \cdot n \le 10n \le c_2 \cdot n$, adevărat pentru $c_1 = 1$; $c_2 = 20$ și $n_0 = 1$

- $n \in \Theta(10n)$

  $f(n) = 10n$; $g(n) = n$

  $\Rightarrow \exists c_1, c_2 > 0$; $\exists n_0 \in \mathbb{N}$ a.î. $c_1 \cdot 10n \le n \le c_2 \cdot 10n$, adevărat pentru $c_1 = \frac{1}{10}$; $c_2 = 1$ și $n_0 = 1$.

(2.) Folosiți o notație asimptotică pentru a descrie următoarea mulțime de funcții:

- $\frac{O(n\sqrt{n})}{\Theta(n)}$

Fie: $x(n) \in O(n\sqrt{n}) \Rightarrow \exists c>0, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \ge n_0$

$0 \le x(n) \le c \cdot n\sqrt{n}$ (1)

$y(n) \in \Theta(n) \Rightarrow \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \ge n_0$

$c_1 \cdot n \le y(n) \le c_2 \cdot n \Rightarrow$

$\Rightarrow \frac{1}{c_2 \cdot n} \le \frac{1}{y(n)} \le \frac{1}{c_1 \cdot n}$ (2)

(1) și (2) $\Rightarrow 0 \le \frac{x(n)}{y(n)} \le \frac{c \cdot n\sqrt{n}}{c_1 \cdot n} \Rightarrow$

c'

$\Rightarrow \exists c' > 0$ a.î. $0 \le \frac{x(n)}{y(n)} \le c' \cdot \sqrt{n} \Rightarrow O(\sqrt{n})$

Scanned with CamScanner

- $\frac{\Theta(n)}{O(\log(n))}$

Fie: $x(n) \in \Theta(n) \Rightarrow \exists c_1, c_2 > 0; \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$c_1 \cdot n \leq x(n) \leq c_2(n)$ (1)

$y(n) \in O(\log(n)) \Rightarrow \exists c > 0; \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$0 \leq y(n) \leq c \cdot \log(n) \Rightarrow$

$\Rightarrow \frac{1}{c\log(n)} \leq \frac{1}{y(n)}$ (2)

Dim (1) și (2) $\Rightarrow 0 \leq \frac{c_1 \cdot n}{c\log(n)} \leq \frac{x(n)}{y(n)} \Rightarrow$

$\Rightarrow 0 \leq c' \cdot \frac{n}{\log(n)} \leq \frac{x(n)}{y(n)} \Rightarrow \Omega\left(\frac{n}{\log(n)}\right)$

- $\frac{\Omega(n^3)}{O(n^2)}$ (dim examen)

Fie: $x(n) \in \Omega(n^3) \Rightarrow \exists c_1 > 0; \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$0 \leq c_1 n^3 \leq x(n)$ (1)

$y(n) \in O(n^2) \Rightarrow \exists c_2 > 0; \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$0 \leq y(n) \leq c_2 \cdot n^2 \Rightarrow$

$\Rightarrow \frac{1}{c_2 \cdot n^2} \leq \frac{1}{y(n)}$ (2)

Dim (1) și (2) $\Rightarrow 0 \leq \frac{c_1 \cdot n^3}{c_2 \cdot n^2} \leq \frac{x(n)}{y(n)} \Rightarrow$

$\Rightarrow \exists c' = \frac{c_1}{c_2} > 0$ a.î. $0 \leq c' \cdot n \leq \frac{x(n)}{y(n)} \Rightarrow \frac{x(n)}{y(n)} \in \Omega(n)$.

- $\frac{O(\log(n))}{\Omega(n^2)}$ (dim examen)

Fie: $x(n) \in O(\log(n)) \Rightarrow \exists c_1 > 0; \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$0 \leq x(n) \leq c_1 \log(n)$ (1)

$y(n) \in \Omega(n^2) \Rightarrow \exists c_2 > 0; \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$0 \leq c_2 \cdot n^2 \leq y(n) \Rightarrow$

$\Rightarrow \frac{1}{y(n)} \leq \frac{1}{c_2 \cdot n^2}$ (2)

Scanned with CamScanner

Din (1) și (2) $\Rightarrow 0 \leq \frac{x(n)}{y(n)} \leq \frac{c_1 \cdot \log(n)}{c_2 \cdot n^2} \Rightarrow$ ($\frac{c_1}{c_2} \to c$)

$\Rightarrow \exists c' = \frac{c_1}{c_2} > 0$ a.î. $0 \leq \frac{x(n)}{y(n)} \leq c' \cdot \frac{\log(n)}{n^2} \Rightarrow$

$\Rightarrow \frac{x(n)}{y(n)} \in O\left(\frac{\log(n)}{n^2}\right)$

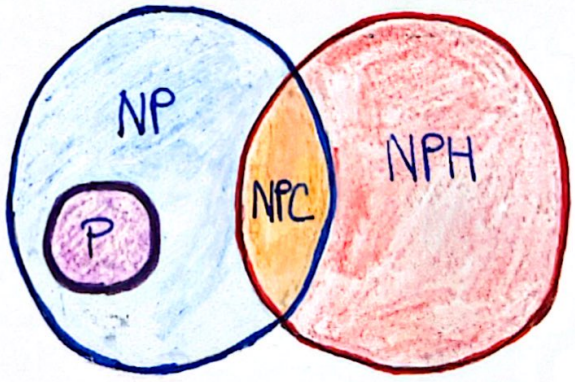
III P, NP, NPH, NPC

P → multimea problemelor pentru care există o soluție deterministic - polinomială.

NP → multimea problemelor pentru care există o soluție nondeterministic - polinomială.

$\boxed{P \subseteq NP}$

- $f \in NPH \overset{def}{\Leftrightarrow} \forall g \in NP,\ g \leq_p f$
- $f \in NPC \overset{def}{\Leftrightarrow} f \in NP$ și $f \in NPH$
- $\forall f \in NP,\ f \leq_p SAT$
- $SAT \in P \Leftrightarrow P = NP$
- $SAT \leq_p g \Leftrightarrow g \in NPH$
- $\left.\begin{matrix} f \in NPH \\ f \leq_p g \end{matrix}\right\} \Rightarrow g \in NPH$
- $f \leq_p g$ și $g \leq_p h \Rightarrow f \leq_p h$
- $\left.\begin{matrix} f \in NPC \\ f \leq_p g \end{matrix}\right\} \Rightarrow g \in NPH$
- $\left.\begin{matrix} f \in NP \\ g \leq_p f \end{matrix}\right\} \Rightarrow g \in NP$
- $\left.\begin{matrix} f \in P \\ g \leq_p f \end{matrix}\right\} \Rightarrow g \in P$



Scanned with CamScanner

Probleme din examene

(1.) Fie f și g două funcții NPH. Dacă știm $f \leq_p g$ și $g \leq_p SAT$, $g \leq_p h$, $h \leq_p f$, ce putem spune despre h?

$f, g \in NPH$
$g \leq_p SAT \Rightarrow g \in NP$ $\Rightarrow g \in NPC$

$f \leq_p g$
$g \leq_p SAT$ $\Rightarrow f \leq_p SAT$
$h \leq_p f$ $\Rightarrow h \leq_p SAT \Rightarrow h \in NP$ (1)

$g \leq_p h$
$g \in NPC$ $\Rightarrow h \in NPH$ (2)

Din (1) și (2) $\Rightarrow h \in NPC$.

(2.) Fie $f \in NP$ și o funcție g; dacă știm că $\forall h \in P, h \leq_p f$ și $f \leq_p g$, ce putem spune despre g?

$f \in NP$
$\forall h \leq_p f$ $\Rightarrow \forall h \in NP$
$h \leq_p f$
$f \leq_p g$ $\Rightarrow h \leq_p g$ $\Rightarrow g \in NPH$

(3.) Fie f și g două probleme NP. Dacă există o soluție deterministic polinomială pentru f și $g \leq_p f$, ce putem spune despre cele două funcții?

„există o soluție deterministic polinomială pentru f" $\Rightarrow$
$f \in P$
$g \leq_p f$ $\Rightarrow g \in P$

Scanned with CamScanner

IV Recurențe

• Teorema Master

Se poate aplica pe recurențe de forma

$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$; $a \geq 1$; $b > 1$.

Cazuri:

I $f(n) = \overset{O}{\Theta}(n^c)$; $c < \log_b a$

$\Rightarrow T(n) = \Theta(n^{\log_b a})$

II $f(n) = \Theta(n^c \cdot \log^k n)$; $k \geq 0$; $c = \log_b a$

$\Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

III $f(n) = \overset{\Omega}{\Theta}(n^c)$; $c > \log_b a$

$\Rightarrow T(n) = \Theta(f(n))$

Exemplu: $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$ (Merge Sort)

$c = 1$

$\log_b a = \log_2 2 = 1$ $\Big|$ $\Rightarrow$ cazul II $\Rightarrow$

$\Rightarrow T(n) = \Theta(n \log n)$

• Metoda Arborelui

→ arbore cu noduri $T(n)$

→ $T(n)$ are a copii de forma $T\left(\frac{n}{b}\right)$

→ frunzele sunt $T(1)$ sau $T(0)$

$T(n) = \sum$ costurilor pe fiecare nivel.

Exemplu: $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$

nivel		cost
0	$T(n)$	n nivel 0 $\Rightarrow n$
1	$T(n/2)$ $T(n/2)$	$2\frac{n}{2} \cdot n$ nivel 1 $\Rightarrow \frac{n}{2}$
2	$T(n/4)$ $T(n/4)$ $T(n/4)$ $T(n/4)$	n nivel 2 $\Rightarrow \frac{n}{2^2}$
⋮	⋮	⋮
h	$T(1)$ $T(1)$ ..... $T(1)$	n nivel $h \Rightarrow 1 = \frac{n}{2^h} \Rightarrow h = \log_2 n$

Scanned with CamScanner

$$T(n) = \sum_{i=0}^{k} \text{cost_nivel_}i = \sum_{i=0}^{k} n = n(k+1) =$$
$$= n(\log_2 n + 1)$$
$$\Rightarrow T(n) \in \Theta(n \log n)$$

- Metoda Substituției

1) intuim complexitatea
2) demonstrație prin inducție
   i) caz de bază : $T(1)$ sau $T(0)$
   ii) caz general
      - ipoteza inductivă : presupunem adev $T(\frac{n}{2})$
      - pasul de inducție: dem. adev pt $T(n)$.

Exemplu: $T(n) = 2T(\frac{n}{2}) + n$

Intuim că $T(n) \in \Theta(n \log n) \Leftrightarrow \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N}$ a.ș.
$\forall n \geq n_0$, $c_1 n \log n \leq T(n) \leq c_2 n \log n$ (*)

Caz de bază : $c_1 \cdot 0 \leq T(1) \leq c_2 \cdot 0$

Caz general :

(ii) Presupunem (*) adevărată pt $T(\frac{n}{2})$
$$c_1 \cdot \frac{n}{2} \log\left(\frac{n}{2}\right) \leq T\left(\frac{n}{2}\right) \leq c_2 \cdot \frac{n}{2} \log\left(\frac{n}{2}\right) \mid \cdot 2$$

(pi) Demonstrăm rel (*) adev pt $T(n)$
$$c_1 \cdot n \log\left(\frac{n}{2}\right) \leq 2T\left(\frac{n}{2}\right) \leq c_2 \cdot n \log\left(\frac{n}{2}\right) \mid + n$$
$$c_1 n \log n - c_1 n \log 2 + n \leq 2T\left(\frac{n}{2}\right) + n \leq c_2 n \log n - c_2 n \log 2 + n$$
$$c_1 n \log n - c_1 n + n \leq T(n) \leq c_2 n \log n - c_2 n + n$$

Vreau să arăt : $\begin{cases} c_1 n \log n \leq c_1 n \log + n(1 - c_1) \\ c_2 n \log n + n(1 - c_2) \leq c_2 n \log n \end{cases}$ $\Leftrightarrow$

$\Leftarrow \begin{cases} 0 \leq n(1 - c_1) \\ n(1 - c_2) \leq 0 \end{cases} \Leftarrow \begin{cases} c_1 \leq 1 \\ c_2 \geq 1 \end{cases}$ $\Rightarrow$ Pot găsi $c_1 = 1$ și $c_2 = 3$ care îndeplinesc condițiile.

$\Rightarrow T(n) \in \Theta(n \log n)$

Scanned with CamScanner

Laborator

(1.) Folosiți Teorema Master pentru a rezolva următoarele recurențe:

- $T_1(n) = 4T_1\left(\frac{n}{4}\right) + 1$

$c = 0$
$\log_b a = \log_4 4 = 1$ $\Rightarrow c < \log_b a \Rightarrow$ cazul I $\Rightarrow$

$\Rightarrow T_1(n) = \Theta(n^{\log_b a}) = \Theta(n)$

- $T_2(n) = 4T_2\left(\frac{n}{2}\right) + n^2$

$c = 2$
$\log_b a = \log_2 4 = 2$ $\Rightarrow c = \log_b a \Rightarrow$ cazul II $\Rightarrow$

$\Rightarrow T_2(n) = \Theta\left(n^{\log_b a} \cdot \log^{k+1} n\right) = \Theta(n^2 \log n)$

- $T_3(n) = 8T_3\left(\frac{n}{2}\right) + 2n^4$

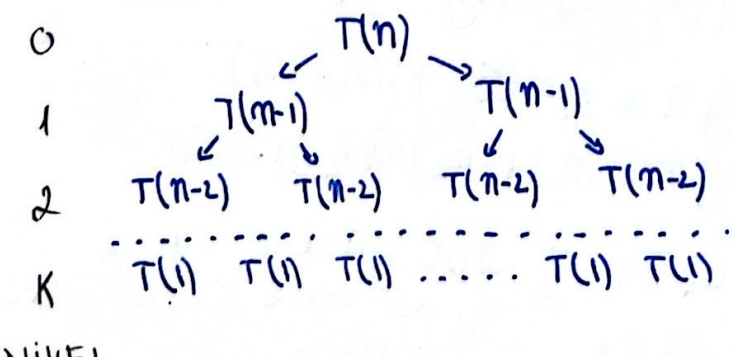
$c = 4$
$\log_b a = \log_2 8 = 3$ $\Rightarrow c > \log_b a \Rightarrow$ cazul III $\Rightarrow$

$\Rightarrow T_3(n) = \Theta(f(n)) = \Theta(n^4)$

(2.) Folosiți cele 3 metode (metoda arborilor, metoda substituției, ~~teorema~~ teorema Master) pentru a rezolva recurențele:

- $T_a(n) = 2T_a(n-1) + 1$ (nu se poate aplica Master)

Metoda arborilor



1 nivel 0: $n-0$
2 nivel 1: $n-1$
4 nivel 2: $n-2$
2^K :
COST nivel K: $n-K=1 \Rightarrow$
$\Rightarrow K = n-1$

Scanned with CamScanner

$$T(n) = \sum_{i=0}^{k} \text{cost_nivel_}i = \sum_{i=0}^{k} 2^i = \frac{2^n - 1}{2-1} = 2^k = 2^{n-n}$$

$\Rightarrow T(n) \in \Theta(2^n)$

Metoda substituției

Intuim că $T(n) \in \Theta(2^n) \Leftrightarrow \exists c_1, c_2 \in \mathbb{R}_+ . \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

se adăuga o ct. și merge $\boxed{c_1 \cdot 2^n \leq T(n) \leq c_2 \cdot 2^n}$ (*)

Caz de bază: $2c_1 \leq T(1) \leq 2c_2$

Caz general:

(ii) Presupunem (*) adevărată pt $T(n-1)$

$$c_1 \cdot 2^{n-1} \leq T(n-1) \leq c_2 \cdot 2^{n-1} \quad | \cdot 2$$

(pi) Demonstrăm relația (*) adev pt $T(n)$

$$c_1 \cdot 2^n \leq 2T(n-1) \leq c_2 \cdot 2^n \quad | +1$$

$$c_1 \cdot 2^n \leq 2T(n-1) + 1 \leq c_2 \cdot 2^n + 1$$

$$c_1 \cdot 2^n + 1 \leq T(n) \leq c_2 \cdot 2^n + 1.$$

Vreau să arăt: $\begin{cases} \cancel{c_1 \cdot 2^n} \leq \cancel{c_1 \cdot 2^n} + 1 \\ \cancel{c_2 \cdot 2^n} + 1 \leq \cancel{c_2 \cdot 2^n} \end{cases} \Rightarrow \begin{cases} 0 \leq 1 \\ 1 \leq 0 \text{ ???} \end{cases}$

- $T_b(n) = T_b(n-1) + 1$ (nu se poate aplica Master)

Metoda arborilor

NIVEL	COST	
0	$T(n)$	1
	↓	
1	$T(n-1)$	1
	↓	
2	$T(n-2)$	1
	↓	
	...	
K	$T(1)$	1

nivel 0: $n-0$

nivel 1: $n-1$

nivel 2: $n-2$

⋮

nivel K: $n-K=1 \Rightarrow$

$\Rightarrow K = n-1$

Scanned with CamScanner

$$T(n) = \sum_{i=0}^{k} \text{cost_nivel_}i = \sum_{i=0}^{k} 1 = 1(k+1) = 1(\cancel{k} - n - 1 + 1) =$$

$$= n$$

$$\Rightarrow T(n) \in \Theta(n)$$

Metoda substituției

Intuim că $T(n) \in \Theta(n) \Rightarrow \exists c_1, c_2 \in \mathbb{R}_+, \exists n_0 \in \mathbb{N}$ a.î.
$\forall n \geq n_0$ avem $c_1 \cdot n \leq T(n) \leq c_2 \cdot n$. (*)

Caz de bază: $c_1 \leq \cancel{T(n)} \; T(1) \leq c_2$

Caz general:

(ii) Presupunem (*) adev pt $T(n-1)$:

$$c_1(n-1) \leq T(n-1) \leq c_2(n-1) \;|+1$$

(ei) Demonstram relația (*) adev pt $T(n)$:

$$c_1 n - c_1 + 1 \leq T(n-1) + 1 \leq c_2 n - c_2 + 1$$

$$c_1 n - c_1 + 1 \leq T(n) \leq c_2 n - c_2 + 1$$

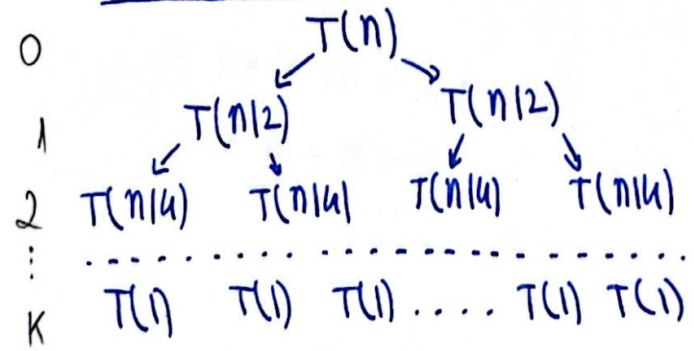
Vreau să arăt: $\begin{cases} \cancel{c_1 n} - c_1 + 1 \geq \cancel{c_1 n} \\ \cancel{c_2 n} - c_2 + 1 \leq \cancel{c_2 n} \end{cases} \Rightarrow \begin{cases} c_1 \leq 1 \\ c_2 \geq 1 \end{cases} \Rightarrow$

$\Rightarrow$ Pot găsi $c_1 = 1$ și $c_2 = 3$ care îndeplinesc condițiile.

$\Rightarrow T(n) \in \Theta(n)$.

- $T_c(n) = 2T_c\left(\frac{n}{2}\right) + \log n$ (nu se poate aplica Master)

Metoda arborilor



$\log n$	nivel 0: n
$2\log\frac{n}{2}$	nivel 1: $\frac{n}{2^1}$
$4\log\frac{n}{4}$	nivel 2: $\frac{n}{2^2}$
$\vdots$	$\vdots$
$2^k \log\frac{n}{2^k}$	nivel k: $\frac{n}{2^k} = 1 \Rightarrow$
COST	$\Rightarrow k = \log_2 n$

$2n$

Scanned with CamScanner

$$T(n) = \sum_{i=0}^{K} \text{cost_nivel_}i = \sum_{i=0}^{K} 2^i \log \frac{n}{2^i} =$$

$$= \sum_{i=0}^{K} 2^i (\log n - \log 2^i) = \sum_{i=0}^{K} 2^i \log n - \sum_{i=0}^{K} i \cdot 2^i =$$

$$= \log n \cdot \sum_{i=0}^{K} 2^i - 2 - 2^{K+1}(K-1)$$

$$= \log n \left(2^{K+1} - 1\right) - 2 - 2^{K+1}(K-1)$$

$$= \log_2 n \left(2 \cdot 2^{\log_2 n} - 1\right) - 2 - 2 \cdot 2^{\log_2 n}(\log_2 n - 1)$$

$$= \log_2 n (2n - 1) - 2 - 2n(\log_2 n - 1)$$

$$= \cancel{2n \cdot \log_2 n} - \log_2 n - 2 - \cancel{2n \log_2 n} + 2n =$$

$$= 2n - \log_2 n - 2$$

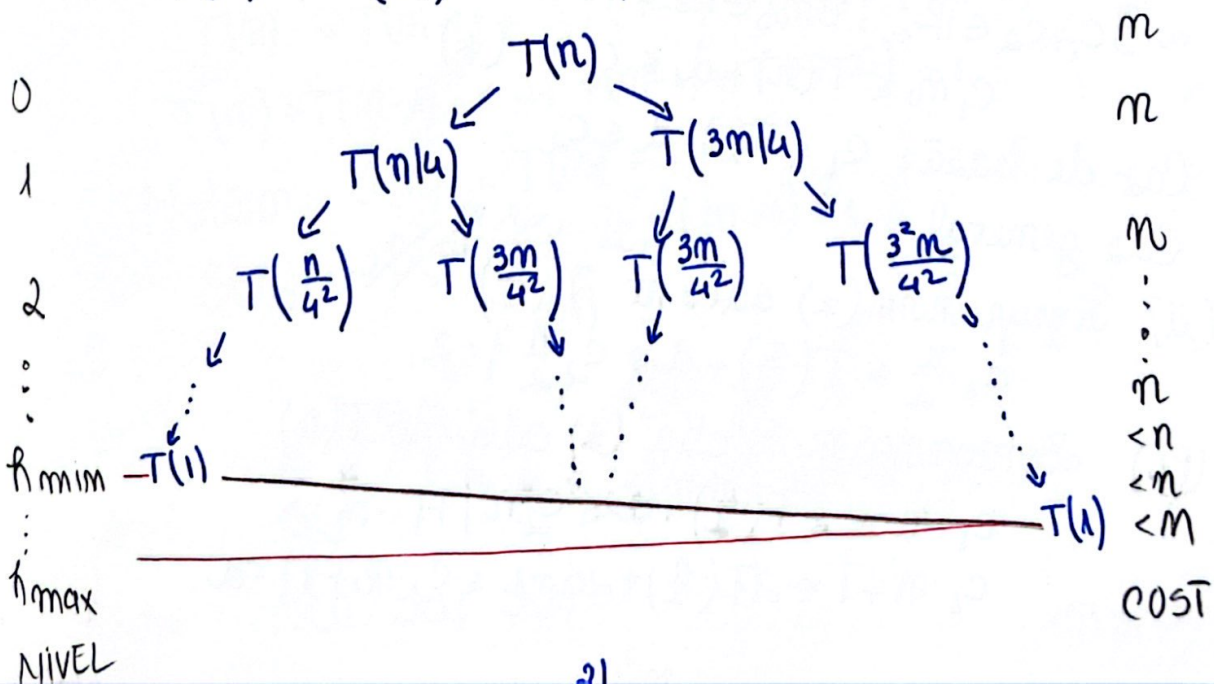
$$\Rightarrow T(n) \in \Theta(2n - \log n - 2) \Rightarrow T(n) \in \Theta(n)$$

Metoda substituției

Intuim $T(n) \in \Theta(n)$ ... (nu mai fac că iar o iau razna)

③ Rezolvați următoarea recurență folosind metoda arborilor:

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n$$



Scanned with CamScanner

$\frac{n}{4^{h_{min}}} = 1 \Rightarrow h_{min} = \log_4 n$

$\frac{n}{(4/3)^{h_{max}}} = 1 \Rightarrow h_{max} = \log_{\frac{4}{3}} n$

$S_{min} \leq T(n) \leq S_{max}$

$S_{min} = \sum_{i=0}^{h_{min}} n = n(1 + h_{min}) = n\log_4 n + n \in \Theta(n \log n)$

$S_{max} = \sum_{i=0}^{h_{max}} n = n(1 + h_{max}) = n\log_{4/3} n + n \in \Theta(n \log n)$

$\Rightarrow T(n) \in \Theta(n \log n)$

(4.) Rezolvați următoarea recurență folosind metoda substituției:

$$T(n) = 2T\left(\frac{n}{2}\right) + 1$$

Master: $c = 0$; $\log_b a = \log_2 2 = 1 \Rightarrow c < \log_b a \Rightarrow$

$\Rightarrow$ caz I $\Rightarrow T(n) = \Theta(n^{\log_b a}) = \Theta(n)$

Metoda substituției

Intuim că $T(n) \in \Theta(n) \Rightarrow T(n) + d \in \Theta(n) \Rightarrow$ (↳ constantă)

$\Rightarrow \exists c_1, c_2 \in \mathbb{R}_+, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$$c_1 n \leq T(n) + d \leq c_2 n \quad (*)$$

Caz de bază: $c_1 \leq T(1) \cdot d \leq c_2$

Caz general:

(ii) Presupunem $(*)$ adev pt $T\left(\frac{n}{2}\right)$

$$c_1 \frac{n}{2} \leq T\left(\frac{n}{2}\right) + d \leq c_2 \frac{n}{2} \mid \cdot 2$$

(Pi) Demonstrăm relația $(*)$ adev pt $T(n)$

$$c_1 \cdot n \leq 2T\left(\frac{n}{2}\right) + d \cdot 2 \leq c_2 n \mid +1$$

$$c_1 \cdot n + 1 \leq 2T\left(\frac{n}{2}\right) + 2d + 1 \leq c_2 \cdot n + 1 \mid -d$$

Scanned with CamScanner

$c_1 \cdot n + 1 - d \le 2T\left(\frac{n}{2}\right) + 1 + d \le c_2 \cdot n + 1 - d$

$c_1 \cdot n + 1 - d \le T(n) + d \le c_2 \cdot n + 1 - d$

Vreau să arăt: $\begin{cases} c_1 n + 1 - d \ge c_1 n \\ c_2 n + 1 - d \le c_2 n \end{cases} \Rightarrow \begin{cases} d \le 1 \\ d \ge 1 \end{cases} \Rightarrow d = 1$

$\Rightarrow T(n) \in \Theta(n)$.

- PROBLEMĂ DE LA EXAMEN

Determinați complexitatea următoarei recurențe:

$$T(n) = 2T(\sqrt{n}) + \log n$$

Notăm: $n = 2^x \Rightarrow x = \log_2 n$

$\Rightarrow T(2^x) = 2T\left(2^{\frac{x}{2}}\right) + x \Rightarrow S(x) = 2S\left(\frac{x}{2}\right) + x$

Notăm: $S(x) = T(2^x)$

$\log_b a = \log_2 2^a = 1$

$x^k = \Theta(x^1 \cdot \log^0 x) \Rightarrow c = 1; k = 0 \Rightarrow c = \log_b a \Rightarrow$

$\Rightarrow$ Th. Master cazul II $\Rightarrow S(x) = \Theta(x^{\log_b a} \cdot \log^{k+1} x) \Rightarrow$

$\Rightarrow S(x) = \Theta(x \log x) \Rightarrow$

$\Rightarrow T(2^x) = \Theta(x \log x) \Rightarrow T(n) = \Theta(\log n \cdot \log(\log n))$

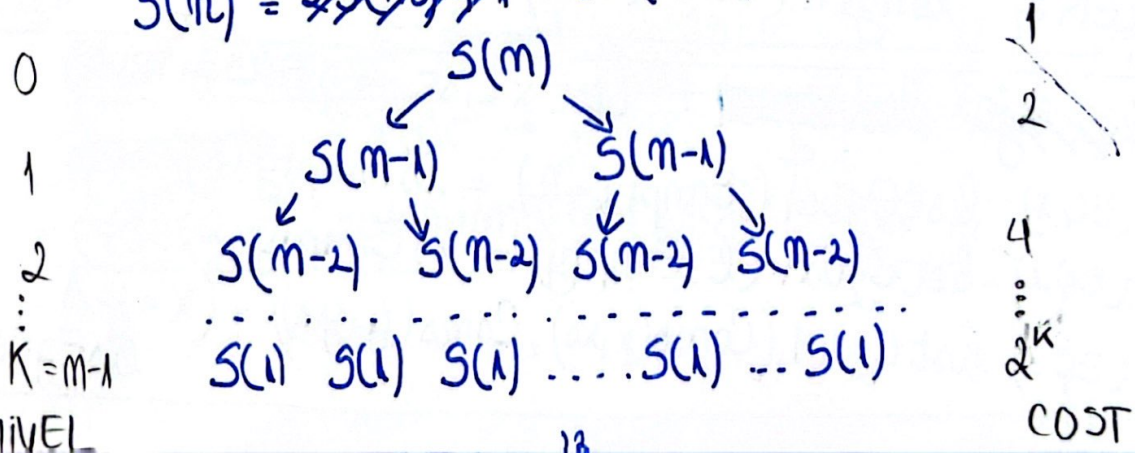
- Determinați complexitatea următoarei recurențe:

$T(n) = T(n-1) + 2T(n-2) + 1 \quad | + T(n-1)$

$T(n) + T(n-1) = 2(T(n-1) + T(n-2)) + 1$

Notăm: $S(n) = T(n) + T(n-1)$

$S(n) = 2S(n-1) + 1$



Scanned with CamScanner

$S(n) = \sum_{i=0}^{n} 2^i = 2^k - 1 = 2^{n-1} - 1$

$\Rightarrow S(n) \in \Theta(2^n)$

$T(n) = S(n) - T(n-1)$

$T(n) \leq S(n) \Rightarrow T(n) \in O(S(n)) = O(2^n)$

V. Algebraic Data Types (TDAs)

1. The „List"

- are 2 constructori

Empty : List

Cons : $\mathbb{N} \times List \rightarrow List$

Lista [1, 2, 5] se exprimă astfel

Cons(1, Cons(2, Cons(3, Empty)))

(Mai ușor de înțeles de la dreapta la stânga)

Operatori

---

isEmpty : List → {TRUE, FALSE} ← nume, domeniu codomeniu

(iE1) isEmpty(Empty) = TRUE

(iE2) isEmpty(Cons(x, xs)) = FALSE } ← axiome

---

length : List → $\mathbb{N}$

(LEN 1) length(~~Empty~~ Empty) = 0

(LEN 2) length(Cons(x, xs)) = 1 + length(xs)

~~listEgal~~ listEqual : List × List → {FALSE, TRUE}

(EQ1) listEqual(Empty, l) = isEmpty(l)

(EQ2) listEqual(l, Empty) = isEmpty(l)

(EQ3) listEqual(Cons(x, xs), Cons(y, ys)) = (x = y ∧ listEqual(xs, ys

---

Scanned with CamScanner

append : List x List → List

(APP1) $\forall l \in List$, append (Empty, l) = l

(APP2) $\forall l \in List$, append (Cons(x, xs), l) = Cons (x, append(xs, l))

---

② Booleans

Constructori : $\frac{}{False : Bool}$ $\quad$ $\frac{}{True : Bool}$

Se pot defini operațiile familiare:

and : Bool x Bool → Bool

(AND1) : and (False, False) = False

(AND2) : and (False, True) = False

(AND 3) : and (True, False) = False

(AND4) : and (True, True) = True

---

③ Tuples

Constructor : $\frac{}{P : \mathbb{N} \times \mathbb{N} \to Pair}$

Se definesc getteri pentru fiecare element.

first : Pair → ℕ

(FST) : first (Pair (x, y)) = x

---

second : Pair → ℕ

(SND) : second (Pair (x, y)) = y

Se definesc și operațiile basic:

pairSum : Pair → ℕ

(PSUM) : pairSum (Pair (x, y)) = x + y

Scanned with CamScanner

④ Natural numbers

Constructori: zero : Natural
succ : Natural → Natural

Logic: 1 = succ(zero) și 2 = succ(succ(zero)).
Putem defini operațiile aritmetice:

add : Natural × Natural → Natural
(ADD1) : add(zero, n) = n
(ADD2) : add(succ(m), n) = succ(add(m, n))

mult : Natural × Natural → Natural
(MUL1) : mult(zero, n) = zero
(MUL2) : mult(succ(m), n) = add(n, mult(m, n)).

⑤ Binary Tree

Constructori : Nil : BTree
Node : ℕ × BTree × BTree → BTree

isNil : BTree → {FALSE, TRUE}
(IN1) isNil(Nil) = TRUE
(IN2) isNil(Node(x, l, r)) = FALSE

size : BTree → ℕ
(SZ1) size(Nil) = 0
(SZ2) size(Node(x, l, r)) = 1 + size(l) + size(r)

height : BTree → ℕ
(H1) : height(Nil) = 0
(H2) : height(Node(x, l, r)) = 1 + max(~~size~~(l), ~~size~~(r))
height height

Scanned with CamScanner

(6) Queue

Constructori : QEmpty : Queue
Enqueue : $\mathbb{N} \times Queue \to Queue$

peak : $Queue \to \mathbb{N}$
(PK1) peak(Enqueue(x, QEmpty)) = x
(PK2) peak(Enqueue(x, Enqueue(y, ys))) = peak(Enqueue(y, ys))

dequeue : $Queue \to Queue$
(DQ1) dequeue(Enqueue(x, QEmpty)) = QEmpty
(DQ2) dequeue(Enqueue(x, xs)) = Enqueue(x, dequeue(xs))

(7) Map

Constructori : MEmpty : Map
Insert : $\mathbb{N} \times \mathbb{N} \times Map \to Map$

exists : $Map \times \mathbb{N} \to \{FALSE, TRUE\}$
(EX1) : exists(Insert(K, V, MEmty), K') = FALSE
(EX2) : exists(Insert(K, V, m), K') = if K' = K then TRUE
else exists(K', m)

get : $Map \times \mathbb{N} \to \mathbb{N}$
(GT1) : get(Insert(K, V, m), K') = if K' = K then V esle
get(m, K')

Laborator

(1.) Definiți axiome pentru următorii operatori de tipul List:

- append (concatenarea unei liste la alta)
- reverse (inversează elementele într-o listă)

Scanned with CamScanner

append : List x List → List

(APP1) append(Empty, l) = l

(APP2) append(Cons(x, xs), l) = Cons(x, append(xs, l))

---

reverse : List → List

(REV1) : reverse(Empty) = Empty

(REV2) : reverse(Cons(x, xs)) = append(reverse(xs), Cons(x, Empty))

---

② Definiți axiome pentru următorii operatori de tipul BTree:

- mirror (arborele oglindit pe verticală)
- flatten (listă cu elementele din arbore)

---

mirror : BTree → BTree

(MRR1) : mirror(Nil) = Nil

(MRR2) : mirror(Node(x, l, r)) = Node(x, mirror(r), mirror(r))

---

flatten : BTree → List

(FLT1) : flatten(Nil) → Empty

(FLT2) : flatten(Node(x, l, r)) = Cons(x, append(flatten(l), flatten(r)))

---

③ Definiți axiome pentru următorii operatori de tipul Map:

- update (un Map cu o nouă asociere cheie - element)
- delete (șterge cheia și valoarea asociată)

Scanned with CamScanner

update : Map × K × V → Map

(U1) update(MEmpty, K, V) = Insert(K, V, MEmpty)

(U2) update(Insert(K, V, m), K', V') = if K = K' then
Insert(K, V', m) else
Insert(K, V, update(m, K', V'))

---

delete : Map × K → Map

(D1) delete(MEmpty, K) = MEmpty

(D2) delete(Insert(K, V, m), K') = if K = K' then m
else Insert(K, V, delete(m, K')).

Probleme de prim examene

1. Definiți un TDA care să modeleze un arbore în care:
   - fiecare nod are doi copii (nod intern), fie zero (frunză);
   - niciun nod intern nu conține o valoare, toate frunzele conțin o valoare.

Leaf : ℕ → BTree

INode : BTree × BTree → BTree

2. Definiți axiome pentru un operator evenPos care extrage dintr-o listă doar elementele de pe pozițiile pare (indexarea începe de la 0).

evenPos : List → List

(EP1) evenPos(Empty) = Empty

(EP2) evenPos(Cons(x, Empty)) = Cons(x, Empty)

(EP3) evenPos(Cons(x, Cons(y, ys))) = Cons(x, evenPos(ys))

Scanned with CamScanner

3. Scrieți axiomele pentru un operator

sumChildren : BTree → BTree

care înlocuiește valoarea fiecărui nod non-frunză cu suma valorilor copiilor în mod recursiv.

sumChildren : BTree → BTree

(SC1) : sumChildren (Nil) = Nil

(SC2) : sumChildren (Node(x, Empty, Empty)) =
= Node (x, Empty, Empty)

(SC3) : sumChildren (Node(x, l, r)) =
= Node (x + sum(sumChildren(l)) + sum(sumChildren(r)),
sumChildren(l), sumChildren(r))

sum : BTree → ℕ

(S1) sum (Nil) = 0

(S2) sum (Node(x, l, r)) = x + sum(l) + sum(r)

4. Definiți axiome pentru un operator oddPos care extrage dintr-o listă doar elementele de pe poziții impare (indexarea începe de la 0).

oddPos : List → List

(OP1) oddPos (Empty) = Empty

(OP2) oddPos (Cons(x, Cons(y, Empty))) = Cons(y, Empty)

(OP3) oddPos (Cons(x, Cons(y, ys))) = Cons(y, oddPos(ys)).

Scanned with CamScanner

VI Inducție structurală

Demonstrații din curs:

- add(n, zero) = n

Caz de bază: n = zero ⇒ add(zero, zero) $\overset{ADD1}{=}$ zero.

Pas inductiv: Presupunem că add(n, zero) = n.

Trebuie să demonstrăm add(Succ(n), zero) = Succ(n)

add(Succ(n), zero) $\overset{ADD2}{=}$ Succ(add(n, zero)) $\overset{IH}{=}$ Succ(n) (Adevărat)

- add(n, Succ(m)) = Succ(add(n, m))

Caz de bază: n = zero ⇒ add(zero, Succ(m)) $\overset{ADD1}{=}$ Succ(m) $\overset{ADD1}{=}$ Suc(Add(zero, m))

Pas inductiv: Presupunem add(n, Succ(m)) = Succ(add(n, m)) adevărată. Trebuie să demonstrăm:

add(Succ(n), Succ(m)) = Succ(add(Succ(n), m))

add(Succ(n), Succ(m)) $\overset{ADD2}{=}$ Succ(add(n, Succ(m))) $\overset{IH}{=}$

= Succ(Succ(add(n, m))) $\overset{ADD2}{=}$ Succ(add(Succ(n), m)) (Adevărat).

- append(l, Empty) = l

Caz de bază: l = Empty ⇒ append(Empty, Empty) $\overset{APP1}{=}$ Empty

Pas inductiv:

Presupunem append(l, Empty) = l. Trebuie să demonstrăm: append(Cons(x, l), Empty) = Cons(x, l).

append(Cons(x, l), Empty) $\overset{APP2}{=}$ Cons(x, append(l, Empty)) $\overset{IH}{=}$ Cons(x, l) (Adevărat).

Scanned with CamScanner

- $height(t) \leq size(t)$

Caz de bază:

$t = Nill \Rightarrow height(Nill) \overset{H1}{=} 0 \overset{Sz1}{=} size(Nill)$

Pas inductiv:

Presupunem $height(l) \leq size(l)$ și $height(r) \leq size(r)$. Trebuie să demonstrăm $height(Node(e,l,r)) \leq size(Node(e,l,r))$.

$height(Node(e,l,r)) \overset{H2}{=} 1 + max(height(l), height(r)) \leq$

$\leq 1 + max(size(l), size(r)) \overset{Sz2}{=} size(Node(e,l,r))$

(Adevărat)

Laborator

Demonstrați următoarele propoziții, folosind inducție structurală.

(1.) $\forall t \in BTree$, $size(t) = size(mirror(t))$.

Caz de bază:

$t = Nill \Rightarrow size(t) \overset{Sz1}{=} 0$

$\overset{M1}{\Rightarrow} mirror(t) = Nill \overset{Sz1}{\Rightarrow} size(mirror(t)) = 0$

Pas inductiv:

Presupunem $size(l) = size(mirror(l))$ și $size(r) = size(mirror(r))$. Trebuie să demonstrăm:

$size(Node(e,l,r)) = size(mirror(Node(e,l,r)))$

$size(Node(e,l,r)) \overset{Sz2}{=} 1 + size(l) + size(r) =$

$= 1 + size(r) + size(l) \overset{IH}{=}$

$= 1 + size(mirror(r)) + size(mirror(l)) \overset{Sz2}{=}$

$= size(Node(e, mirror(r), mirror(l))) \overset{M2}{=}$

$= size(mirror(Node(e,l,r)))$ (Adevărat)

Scanned with CamScanner

② $\forall t \in BTree$, $size(t) = length(flatten(t))$

Caz de bază

$t = NIL \overset{SZ1}{\Rightarrow} size(t) = 0$

$\overset{F1}{\Rightarrow} flatten(t) = Empty \overset{L1}{\Rightarrow} length(flatten(t)) = 0$

Pas inductiv

Presupunem $size(e) = length(flatten(e))$ și $size(r) = length(flatten(r))$. Trebuie să demonstrăm:

$size(Node(e,l,r)) = length(flatten(Node(e,l,r)))$.

$size(Node(e,l,r)) \overset{SZ2}{=} 1 + size(e) + size(r) \overset{IH}{=}$

$= 1 + length(flatten(e)) + length(flatten(r)) \overset{*}{=}$

$= 1 + length(append(flatten(e), flatten(r))) \overset{L2}{=}$

$= length(Cons(e, append(flatten(e), flatten(r)))) \overset{F2}{=}$

$= length(flatten(Node(e,l,r)))$ (Adevărat).

* $length(append(a,b)) = length(a) + length(b)$

Caz de bază

$a = Empty \Rightarrow length(append(Empty, b)) \overset{APP1}{=}$

$= length(b)$

$a = Empty \overset{L1}{=} length(a) = 0 \Rightarrow$

$\Rightarrow length(a) + length(b) = length(b)$

Pas inductiv

Presupunem $length(append(a,b)) = length(a) + length(b)$.

Trebuie să demonstrăm:

$length(append(Cons(x,a), b)) = length(Cons(x,a)) + length(b)$

$length(append(Cons(x,a), b)) \overset{APP2}{=}$

$= length(Cons(x, append(a,b))) \overset{L2}{=}$

$= 1 + length(append(a,b)) \overset{IH}{=}$

$= 1 + length(a) + length(b) \overset{L2}{=}$

$= length(Cons(x,a)) + length(b)$ (Adevărat)

Scanned with CamScanner

③ $\forall l \in List$, append(l, Empty) = l

Caz de bază:

l = Empty ⇒ append(Empty, Empty) $\overset{APP1}{=}$ Empty

Pas inductiv:

Presupunem append(l, Empty) = l. Trebuie să demonstrăm: append(Cons(x, l), Empty) = Cons(x, l).

append(Cons(x, l), Empty) $\overset{APP2}{=}$
= Cons(x, append(l, Empty)) $\overset{IH}{=}$
= Cons(x, l) (Adevărat).

④ $\forall l_1, l_2, l_3 \in List$, append($l_1$, append($l_2$, l_3)) =
= append(append(l_1, l_2), l_3)

Caz de bază

l_1 = Empty ⇒ append(Empty, append(l_2, l_3)) $\overset{APP1}{=}$
= append(l_2, l_3).

append(append(Empty, l_2), l_3) $\overset{APP1}{=}$
= append(l_2, l_3)

Pas inductiv

Presupunem append(l_1, append(l_2, l_3)) = append(append(l_1, l_2), l_3). Trebuie să demonstrăm că:

append(Cons(x, l_1), append(l_2, l_3)) = append(append(Cons(x, l_1), l_2), l_3).

append(Cons(x, l_1), append(l_2, l_3)) $\overset{APP2}{=}$
= Cons(x, append(l_1, append(l_2, l_3))) $\overset{IH}{=}$
= Cons(x, append(append(l_1, l_2), l_3)) $\overset{APP2}{=}$
= append(Cons(x, append(l_1, l_2)), l_3) $\overset{APP2}{=}$
= append(append(Cons(x, l_1), l_2), l_3) (Adevărat).

Scanned with CamScanner

⑤ $\forall l_1, l_2 \in List$, length(append($l_1, l_2$)) = length(append($l_2, l_1$))

Caz de bază:

$l_1 = Empty \Rightarrow$ length(append(Empty, l_2)) $\overset{APP1}{=}$

$=$ length(l_2) $\overset{ex3}{=}$ length(append(l_2, Empty)).

Pas inductiv:

Presupunem length(append(l_1, l_2)) = length(append(l_2, l_1)).

Trebuie să demonstrăm:

length(append(Cons(x, l_1), l_2)) = length(append(l_2, Cons(x, l_1)))

length(append(Cons(x, l_1), l_2)) $\overset{APP2}{=}$

$=$ length(Cons(x, append(l_1, l_2))) $\overset{L2}{=}$

$= 1 +$ length(append(l_1, l_2)) $\overset{IH}{=}$ ex2 ⊛

~~$= 1 +$ length(append(l_2, l_1))~~

$= 1 +$ length(l_1) + length(l_2) $=$

$=$ length(l_2) $+ 1 +$ length(l_1) $\overset{L2}{=}$

$=$ length(l_2) + length(Cons(x, l_1)) $\overset{ex2}{=}$ ⊛

$=$ length(append(l_2, Cons(x, l_1))) (Adevărat).

EXERCIȚIU DIN EXAMEN

Demonstrați, prin inducție structurală, proprietatea:

$\forall l \in List$, length(l) = length(reverse(l))

Caz de bază:

$l =$ Empty $\overset{L1}{\Rightarrow}$ length(l) = 0

$\overset{REV1}{\Rightarrow}$ reverse(l) = Empty $\overset{L1}{\Rightarrow}$ ~~absurd~~

length(reverse(l)) = 0

Pas inductiv:

Presupunem length(l) = length(reverse(l)).

Trebuie să demonstrăm:

length(Cons(x, l)) = length(reverse(Cons(x, l)))

Scanned with CamScanner

$length(Cons(x, e)) \overset{L2}{=} 1 + length(e)$

$length(reverse(Cons(x, e))) \overset{REV2}{=}$

$= length(append(reverse(e), Cons(x, Empty))) \overset{TH}{=}$

$= length(reverse(e)) + length(Cons(x, Empty)) \overset{IH}{=}$

$= length(e) + length(Cons(x, Empty)) \overset{L2}{=}$

$= length(e) + 1 + length(Empty) \overset{L1}{=}$

$= length(e) + 1 + 0 =$

$= 1 + length(e)$

Scanned with CamScanner

EXAMEN 2023 V2.

1. Scrieți tabela de tranziții pentru o Mașină Turing cu alfabetul de input $\{0,1\}$, care acceptă șirul primit doar dacă acesta conține și subșirul 10 și subșirul 01. (ordinea în care apar e irelevantă).

	0	1	□
start	found0, 0, →	found1, 1, →	N, □, -
found0	found0, 0, →	found01, 1, →	N, □, -
found1	found10, 0, →	found1, 1, →	N, □, -
found01	found01, 0, →	found01, 1, →	reset01, □, ←
found10	found10, 0, →	found10, 1, →	reset10, □, ←
reset01	reset01, 0, ←	reset01, 1, ←	search10, □, →
reset10	reset10, 0, ←	reset10, 1, ←	search01, □, →
search10	search10, 0, →	found011, 1, →	N, □, -
search01	found100, 0, →	search01, 1, →	N, □, -
found011	Y, 0, -	found011, 1, →	N, □, -
found100	found100, 0, →	Y, 1, -	N, □, -

2. Considerați următoarea mașină Turing:

	0	1	□
start	remember0, □, →	remember1, □, →	Y, □, → -
remember0	remember0, 0, →	remember0, 1, →	expect0, □, ←
remember1	remember1, 0, →	remember1, 1, →	expect1, □, ←
expect0	reset, □, ←	N, 1, -	Y, □, -
expect1	N, 0, -	reset, □, ←	Y, □, -
reset	reset, 0, ←	reset, 1, ←	start, □, →

Scanned with CamScanner

Scrieți istoria computațională a mașinii pentru inputul 010.

$(\varepsilon, 010, start) \vdash_M (\varepsilon, 10, remember0) \vdash_M (1, 0, remember0) \vdash_M$
$\vdash_M (10, \square, remember0) \vdash_M (1, 0, expect0) \vdash_M (\varepsilon, 1, reset) \vdash_M$
$\vdash_M (\varepsilon, \square 1, reset) \vdash (\varepsilon, 1, start) \vdash_M (\varepsilon, \square, remember1)$
$\vdash_M (\varepsilon, \square, expect1) \vdash_M (\varepsilon, \square, Y).$

(3.) Fie funcția

$$f(enc(M_1, M_2)) = \begin{cases} TRUE, & M_1[enc(M_2)] \equiv M_2[enc(M_1)] \\ FALSE, & altfel. \end{cases}$$

Demonstrați, folosind reduceri, că problema este nedecidabilă (i.e. $f \notin R$).

Vom face reducerea $HALT \leq_m f \Big| \Rightarrow f \notin R$
Cunoaștem $HALT \notin R$

$HALT \leq_m f \overset{def}{\Leftrightarrow} \exists t : \Sigma^* \to \Sigma^*$ a.î.

$\begin{cases} t \text{ e computabilă} \\ \forall (M, w),\ HALT(enc((M,w))) = TRUE \Leftrightarrow f(t(enc((M,w)))) = TRUE. \end{cases}$

Transformarea t produce mașinile M_1 și M_2:

$M_1(x)$:
  if($x == enc(M_2)$)
    run $M[w]$
    acceptă

$M_2(x)$:
  acceptă

Transformarea t trebuie doar să reproducă comportamentul celor 2 mașini, deci este computabilă.

Demonstrăm echivalența cu cele 2 implicații:

„$\Rightarrow$": $HALT(enc(M,w)) = TRUE \Rightarrow M[w]$ se oprește $\Rightarrow$
$\Rightarrow$ run $M[w]$ nu ciclează $\Rightarrow$ pentru $x = enc(M_2)$, M_1 tranziționează în Yes; de asemenea M_2 tranziționează în Yes $\Rightarrow M_1[enc(M_2)] \equiv M_2[enc(M_1)] \Rightarrow f(t(enc(M,w))) = TRUE$ (1)

Scanned with CamScanner

$\Leftarrow$: $f(t(enc(M,w))) = TRUE \Rightarrow M_1[enc(M_2)] = M_2[enc(M_2)] \Rightarrow$

$\Rightarrow M_1$ și M_2 au același comportament ; M_2 tranziționează în yes $\Rightarrow M_1$ tranziționează în yes $\Rightarrow M[w]$ nu ciclează

$\Rightarrow M[w]$ se oprește $\Rightarrow HALT(enc(M,w)) = TRUE$ (2)

Din (1), (2) $\Rightarrow HALT \leq_m f \Rightarrow f \notin \mathcal{R}$

(4.) Determinați complexitatea următoarei recurențe:

$$T(n) = 2T(\sqrt{n}) + \log n$$

Notăm: $n = 2^x \Rightarrow x = \log_2 n$

$$T(2^x) = 2T(2^{x/2}) + x \quad \Big| \Rightarrow S(x) = 2S\left(\frac{x}{2}\right) + x$$

Notăm: $S(x) = T(2^x)$

$\log_b a = \log_2 2 = 1 \Big| \Rightarrow c = \log_b a \Rightarrow$ Th Master cazul II $\Rightarrow$

$c = 1 ; k = 0$

$$\Rightarrow S(x) = \Theta\left(x^{\log_b a} \log^{k+1} x\right) = \Theta(x \log x) \rightarrow$$

$$\Rightarrow T(2^x) = \Theta(x \log x) \Rightarrow$$

$$\Rightarrow T(n) = \Theta(\log n \cdot \log(\log n))$$

(5.) Folosiți o notație asimptotică pentru a descrie următoarea mulțime de funcții:

$$\frac{O(\log n)}{\Omega(n^2)}$$

Fie: $x(n) \in O(\log n) \Rightarrow \exists c_1 \in \mathbb{R}_+, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$$0 \leq x(n) \leq c_1 \log n \quad (1)$$

$y(n) \in \Omega(n^2) \Rightarrow \exists c_2 \in \mathbb{R}_+, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$

$$0 \leq c_2 \cdot n^2 \leq y(n) \quad \uparrow^{-1}$$

$$0 \leq \frac{1}{y(n)} \leq \frac{1}{c_2 \cdot n^2} \quad (2)$$

Scanned with CamScanner

Dim (1) și (2) $\Rightarrow 0 \leq \frac{x(n)}{y(n)} \leq \frac{c_1 \cdot \log n}{c_2 \cdot n^2} \Rightarrow$

$\Rightarrow \exists c' = \frac{c_1}{c_2} \in \mathbb{R}_+$ a.î. $0 \leq \frac{x(n)}{y(n)} \leq c' \cdot \frac{\log n}{n^2} \Rightarrow$

$\Rightarrow \frac{x(n)}{y(n)} \in O\left(\frac{\log n}{n^2}\right)$

(6.) Determinați complexitatea următoarei secvențe de cod, folosind o notație Theta:

```
int f(int *arr, int n){
    if(n<=1) return arr[0];
    return f(arr, n/2) + f(arr+n/2, n/2);
}
```

Putem scrie relația de recurență a algoritmului astfel:

$T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + \Theta(1) \Leftrightarrow$

$T(n) = 2T(\frac{n}{2}) + \Theta(1)$

$\log_b a = \log_2 2 = 1$

$f(n) = \Theta(1) = \Theta(n^0) \Rightarrow c = 0$ $\Big| \Rightarrow c < \log_b a \Rightarrow$

$\Rightarrow$ Th Master cazul I $\Rightarrow T(n) = \Theta(n^{\log_b a}) \Rightarrow$

$\Rightarrow T(n) \in \Theta(n)$.

(7.) Fie f și g două funcții NPH. Dacă știm că $f \leq_p g$ și $g \leq_p$ SAT, $g \leq_p h$, $h \leq_p f$, ce putem spune despre h?

$g \leq_p SAT \Rightarrow \begin{matrix} g \in NP \\ g \in NPH \end{matrix} \Big| \Rightarrow g \in NPC$

$\begin{matrix} f \leq_p g \\ g \leq_p SAT \end{matrix} \Big| \Rightarrow \begin{matrix} f \leq_p SAT \\ h \leq_p f \end{matrix} \Big| \Rightarrow h \leq_p SAT \Rightarrow h \in NP$ (1)

$\begin{matrix} g \leq_p h \\ g \in NPC \end{matrix} \Big| \Rightarrow h \in NPH$ (2); Dim (1) și (2) $\Rightarrow h \in NPC$.

Scanned with CamScanner

(8.) Scrieți axiomele pentru un operator
sumChildren : BTree → BTree
care înlocuiește valoarea fiecărui nod frunză cu suma valorilor copiilor în mod recursiv.

sumChildren : BTree → BTree

(SC1) sumChildren(Nil) = Nil

(SC2) sumChildren(Node(x, Nil, Nil)) = Node(x, Nil, Nil)

(SC2) sumChildren(Node(x, l, r)) =

= Node(x + sum(sumChildren(l)) + sum(sumChildren(r)), sumChildren(l), sumChildren(r))

sum : BTree → ℕ

(S1) sum(Nil) = 0

(S2) sum(Node(x, l, r)) = x + sum(l) + sum(r)

(S3) sum(Node(x, Nil, Nil)) = x.

(9.) Demonstrați, folosind inducție structurală, că:
$\forall t \in BTree$, height(t) = height(mirror(t))

Caz de bază

$t = Nil \overset{H1}{\Rightarrow}$ height(t) = 0

$\overset{M1}{\Rightarrow}$ mirror(t) = Nil $\overset{H1}{\Rightarrow}$ height(mirror(t)) = 0

Pas inductiv

Presupunem height(l) = height(mirror(l)) și height(r) = height(mirror(r)). Trebuie să demonstrăm:
height(Node(x, l, r)) = height(mirror(Node(x, l, r))).

height(Node(x, l, r)) $\overset{H2}{=}$

= 1 + max(height(l), height(r)) (1)

Scanned with CamScanner

$height(mirror(Node(x,l,r))) \overset{M2}{=}$

$= height(Node(x, mirror(r), mirror(l))) \overset{M2}{=}$

$= 1 + max(height(mirror(r)), height(mirror(l))) \overset{IH}{=}$

$= 1 + max(height(r), height(l))$ (2)

Din (1) și (2) $\Rightarrow$ Adevărat.

Scanned with CamScanner

Mai multe exerciții :(

(1.) Determinați complexitatea următoarei recurențe:

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{3}\right) + n\log n$$

Avem: $2T\left(\frac{n}{3}\right) + n\log n \leq T(n) \leq 2T\left(\frac{n}{2}\right) + n\log n$

- pentru $2T\left(\frac{n}{3}\right) + n\log n$

$\log_b a = \log_3 2 \mid \Rightarrow c > \log_b a \Rightarrow$
$c = 1 ; k = 1$

$\Rightarrow$ Th Master cazul III $\Rightarrow \Theta(n\log n)$

- pentru $2T\left(\frac{n}{2}\right) + n\log n$

$\log_b a = \log_2 2 = 1 \mid \Rightarrow c = \log_b a \Rightarrow$
$c = 1 ; k = 1$

$\Rightarrow$ Th Master cazul II $\Rightarrow \Theta(n\log^2 n)$???

(2.) Determinați complexitatea următoarei secvențe de cod, folosind o notație Theta:

```
for(i=0; i<n; i++)
    for(j=0; j<n; j++)
        for(k=i; k<j; k++)
            sum += arr[k].
```

Bucla externă (i) execută corpul său de n ori
$\Rightarrow \Theta(n)$

Bucla intermediară (j) execută corpul său de n ori
$\Rightarrow \Theta(n)$

Bucla internă (k) se execută de la i până la j

$i = 0 \Rightarrow$ for-ul se execută de $0+1+2+\dots+n-1 = \frac{(n-1)n}{2}$ ori.

$i = 1 \Rightarrow$ for-ul se execută de $0+0+1+\dots+n-2 = \frac{(n-2)(n-1)}{2}$ ori

.....

$i = n-1 \Rightarrow$ for-ul se execută de 0 ori

Scanned with CamScanner

$\Rightarrow N = \frac{n(n-1)}{2} + \frac{(n-1)(n-2)}{2} + \dots + \frac{[n-(n-1)](n-n)}{2}$

$= \frac{1}{2}\left[1\cdot 2 + 2\cdot 3 + \dots + \frac{(n-1)n}{1}\right] = \frac{(n-1)n(n+1)}{6}$

$\Rightarrow \Theta(n^3)$

Buclele sunt imbricate $\Rightarrow \Theta(n^5)$

(3.) Demonstrați, folosind inducție structurală, că

$\forall t \in BTree$, $mirror(mirror(t)) = t$.

Caz de bază:

$t = Nil \overset{M1}{\Rightarrow} mirror(t) = nil \overset{M1}{\Rightarrow} mirror(mirror(t)) = Nil$.

Pas inductiv:

Presupunem că $mirror(mirror(e)) = e$ și $mirror(mirror(r)) = r$. Trebuie să demonstrăm:

$mirror(mirror(Node(x,e,r))) = Node(x,l,r)$.

$mirror(mirror(Node(x,e,r))) \overset{M2}{=}$

$= mirror(Node(x, mirror(r), mirror(e))) \overset{M2}{=}$

$= Node(x, mirror(mirror(e)), mirror(mirror(r))) \overset{IH}{=}$

$= Node(x, e, r)$ (Adevărat).

(4.) Demonstrați, folosind inducție structurală, că

$\forall t \in BTree$, $size(mirror(t)) = size(t)$.

Caz de bază:

$t = Nil \overset{M1}{\underset{St}{\Rightarrow}} mirror(t) = Nil \overset{St}{\Rightarrow} size(mirror(t)) = 0$

$size(t) = 0$.

Scanned with CamScanner

Pas inductiv:

Presupunem că size(mirror(l)) = size(l) și size(mirror(r)) = size(r). Trebuie să demonstrăm:

size(mirror(Node(x,l,r))) = size(Node(x,l,r))

size(mirror(Node(x,l,r))) $\overset{M2}{=}$

= size(Node(x, mirror(r), mirror(l))) $\overset{Sz2}{=}$

= 1 + size(mirror(r)) + size(mirror(l)) $\overset{iH}{=}$

= 1 + size(r) + size(l) =

= 1 + size(l) + size(r) $\overset{Sz2}{=}$

= size(Node(x,l,r)) (Adevărat)

Scanned with CamScanner